EXPERIMENT NO. 06

Simulated Annealing

1. Problem + Algorithm

Experiment No.: 08

Title: Simulated Annealing

Aim: Implement Simulated Annealing to minimize the given objective function.

Problem Statement: Minimize the objective function $f(x) = x^2 + 10 \sin(x)$ using a temperature-controlled stochastic search.

Algorithm

- Initialize solution.
- Set initial temperature.
- Generate neighboring solution.
- Calculate change in objective function.
- Accept/reject according to probability.
- Reduce temperature.
- Repeat until stopping criterion.
- Return best solution.

2. Program

Key parameter settings

```
x0 = 8
T0 = 100
alpha = 0.95
T_min = 0.001
neighbour_step = 2.0
```

Function definitions

```
def f(x):
    return x**2 + 10 * np.sin(x)
```

Core algorithm

```
def simulated_annealing(x0, T0, alpha, T_min, step=1.0, seed=42, lo=-10, hi=10):
    rng = random.Random(seed)
    x, fx = x0, f(x0)
    best_x, best_f = x, fx
    T = T0
    path = [(x, fx)]
    best_history = [best_f]
    while T >= T_min:
        candidate = min(max(x + rng.uniform(-step, step), lo), hi)
        delta = f(candidate) - fx
        if delta <= 0 or rng.random() < math.exp(-delta / T):
            x, fx = candidate, f(candidate)
            if fx < best_f:
                best_x, best_f = x, fx
        path.append((x, fx))
        best_history.append(best_f)
        T *= alpha
    return best_x, best_f, path, best_history
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function: $f(x) = x^2 + 10 \sin(x)$

Search Interval: $-10 \leq x \leq 10$

Settings

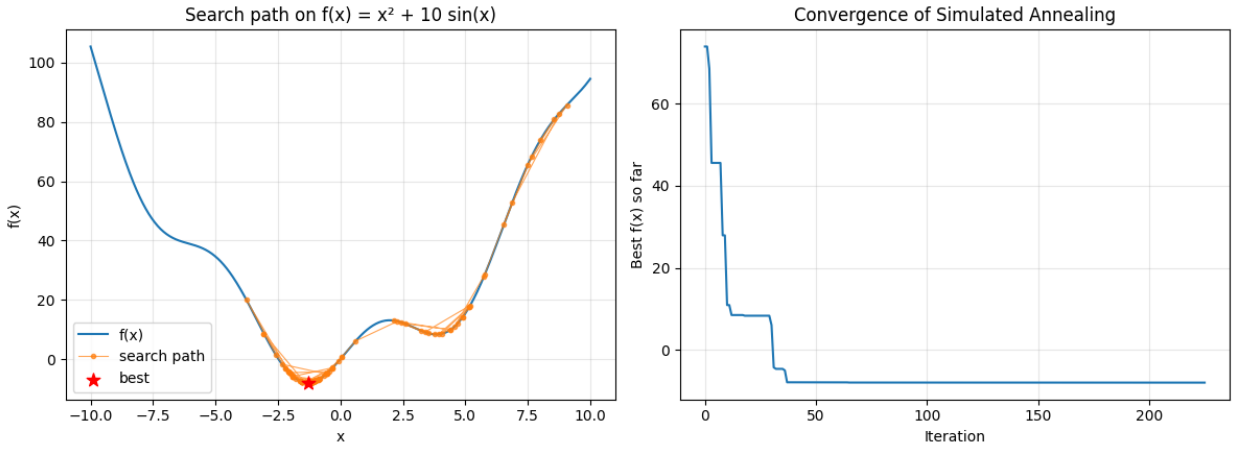
- Initial Solution: x0 = 8
- Initial Temperature: T0 = 100
- Cooling Rate: α = 0.95
- Stopping Temperature: T_min = 0.001 (225 iterations)
- Neighbour Step: ±2.0 (uniform)

Program Execution

```
Iterations     : 225
Best x         : -1.3055
Minimum f(x)   : -7.9458
```

Execution Status: Successfully Executed

Convergence Graph



X-axis → Left: x vs f(x) with search path; Right: Iteration

Y-axis → Left: f(x); Right: Best objective-function value so far

Analysis

The search starts at x = 8 on a high part of the curve, accepts some uphill moves while the temperature is high, and settles in the global basin near x = −1.3 as the temperature falls. The final value is close to the true minimum of about −7.9458.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Initial Temperature T0	100	20	x = −1.3055, f = −7.9458	x = −1.3114, f = −7.9457	Lower T0 means fewer uphill moves early; the global basin was still reached.
2	Cooling Rate alpha	0.95	0.80	x = −1.3055, f = −7.9458	x = 3.8589, f = 8.3175	Fast cooling froze the search in the local minimum near x = 3.86 after only 52 iterations.
3	Stopping Temperature T_min	0.001	0.5	x = −1.3055, f = −7.9458	x = −1.3642, f = −7.9263	Stopping earlier (104 iterations) gave a slightly less refined answer.
4	Initial Solution x0	8	-8	x = −1.3055, f = −7.9458	x = −1.3731, f = −7.9198	Starting from the other side still reached the global basin.

EXPERIMENT NO. 07

Genetic Algorithm for TSP

1. Problem + Algorithm

Experiment No.: 09

Title: Genetic Algorithm for TSP

Aim: Implement Genetic Algorithm (GA) for solving the Traveling Salesman Problem (TSP).

Problem Statement: Find a short closed tour that visits every city exactly once and returns to the starting city. The objective is to minimize total tour distance.

Algorithm

- Initialize a population of valid city permutations.
- Calculate route distance and fitness.
- Select parents using tournament selection.
- Apply ordered crossover to create offspring.
- Apply swap mutation.
- Keep the best individuals using elitism.
- Repeat for the specified number of generations.
- Return the best route found.

2. Program

Key parameter settings

```
rng = np.random.default_rng(7)
n_cities = 20
city_names = list("ABCDEFGHIJKLMNOPQRST")
coords = rng.integers(0, 100, size=(n_cities, 2))
dist = np.linalg.norm(coords[:, None] - coords[None, :], axis=2)

population_size = 100
generations = 300
crossover_rate = 0.9
mutation_rate = 0.2
```

Function definitions

```
def route_distance(route):
    return sum(dist[route[i], route[(i + 1) % n_cities]] for i in range(n_cities))

def fitness(route):
    return 1 / route_distance(route)

def tournament(pop, k=3):
    return min(random.sample(pop, k), key=route_distance)

def ordered_crossover(p1, p2):
    a, b = sorted(random.sample(range(n_cities), 2))
    child = [None] * n_cities
    child[a:b + 1] = p1[a:b + 1]
    rest = [c for c in p2 if c not in child]
    for i in range(n_cities):
        if child[i] is None:
            child[i] = rest.pop(0)
    return child

def swap_mutation(route):
    i, j = random.sample(range(n_cities), 2)
    route[i], route[j] = route[j], route[i]
    return route
```

Core algorithm

```
def genetic_algorithm(pop_size, gens, cx_rate, mut_rate, seed=42, elite=2):
    random.seed(seed)
    pop = [random.sample(range(n_cities), n_cities) for _ in range(pop_size)]
    history = []
    for _ in range(gens):
        pop.sort(key=route_distance)
        history.append(route_distance(pop[0]))
        new_pop = [r[:] for r in pop[:elite]]
        while len(new_pop) < pop_size:
            p1, p2 = tournament(pop), tournament(pop)
            child = ordered_crossover(p1, p2) if random.random() < cx_rate else p1[:]
            if random.random() < mut_rate:
                child = swap_mutation(child)
            new_pop.append(child)
        pop = new_pop
    best = min(pop, key=route_distance)
    history.append(route_distance(best))
    return best, history
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function: Minimize total tour distance; fitness = 1 / total distance

Search Interval: 20 cities A–T with random coordinates in [0, 100] (seed 7)

Settings

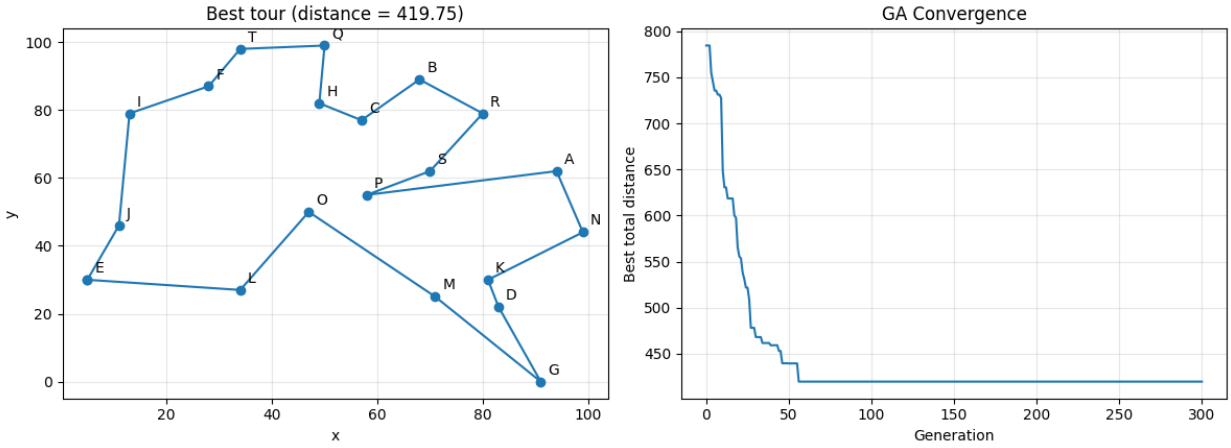
- Initial Solution: Population = 100 random tours
- Crossover Rate: 0.90
- Mutation Rate: 0.20
- Maximum Generations: 300

Program Execution

```
Best Route      : E -> L -> O -> M -> G -> D -> K -> N -> A -> P -> S -> R -> B -> C -> H -> Q -> T -> F -> I -
> J -> E
Minimum Distance: 419.748
Best Fitness    : 0.002382
```

Execution Status: Successfully Executed

Convergence Graph



X-axis → Left: x coordinate; Right: Generation

Y-axis → Left: y coordinate; Right: Best total distance

Analysis

The best distance drops steeply in the first 60 generations as ordered crossover combines good segments and then stays flat once elitism holds the best tour. The best of the initial random tours is about 785 long, while the final GA tour is 419.75, a reduction of about 47%.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Population Size	100	20	Distance = 419.748	Distance = 483.980	Smaller population has less diversity and ended with a longer tour.
2	Generations	300	30	Distance = 419.748	Distance = 468.105	Too few generations; the search stopped before converging.
3	Crossover Rate	0.90	0.50	Distance = 419.748	Distance = 471.736	Less recombination slowed the improvement.
4	Mutation Rate	0.20	0.60	Distance = 419.748	Distance = 413.023	Higher mutation increased exploration and found a shorter tour.

EXPERIMENT NO. 08

Particle Swarm Optimization

1. Problem + Algorithm

Experiment No.: 10

Title: Particle Swarm Optimization

Aim: Implement Particle Swarm Optimization (PSO) to minimize the Rosenbrock objective function.

Problem Statement: Minimize the two-variable Rosenbrock function using a swarm of particles that update their positions and velocities from personal and global best solutions.

Algorithm

- Initialize particle positions and velocities.
- Evaluate the objective function for every particle.
- Set each particle's personal best.
- Set the global best particle.
- Update velocity using inertia, personal and global terms.
- Update and bound particle positions.
- Update personal and global best values.
- Repeat until the maximum number of iterations.
- Return the global best solution.

2. Program

Key parameter settings

```
n_particles = 30
lower = np.array([-2.0, -1.0])
upper = np.array([2.0, 3.0])
w = 0.7
c1 = 1.5
c2 = 1.5
max_iterations = 100
```

Function definitions

```
def rosenbrock(p):
    x, y = p
    return (1 - x)**2 + 100 * (y - x**2)**2
```

Core algorithm

```
def pso(n_particles, w, c1, c2, max_iterations, seed=1):
    rng = np.random.default_rng(seed)
    pos = rng.uniform(lower, upper, (n_particles, 2))
    vel = rng.uniform(-0.2, 0.2, (n_particles, 2))
    pbest = pos.copy()
    pbest_val = np.array([rosenbrock(p) for p in pos])
    g = pbest_val.argmin()
    gbest, gbest_val = pbest[g].copy(), pbest_val[g]
    history = [gbest_val]
    for _ in range(max_iterations):
        r1, r2 = rng.random((2, n_particles, 2))
        vel = w * vel + c1 * r1 * (pbest - pos) + c2 * r2 * (gbest - pos)
        pos = np.clip(pos + vel, lower, upper)
        val = np.array([rosenbrock(p) for p in pos])
        better = val < pbest_val
        pbest[better], pbest_val[better] = pos[better], val[better]
        g = pbest_val.argmin()
        if pbest_val[g] < gbest_val:
            gbest, gbest_val = pbest[g].copy(), pbest_val[g]
        history.append(gbest_val)
    return gbest, gbest_val, history
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function: $f(x,y) = (1-x)^2 + 100(y-x^2)^2$

Search Interval: $-2 \le x \le 2, -1 \le y \le 3$

Settings

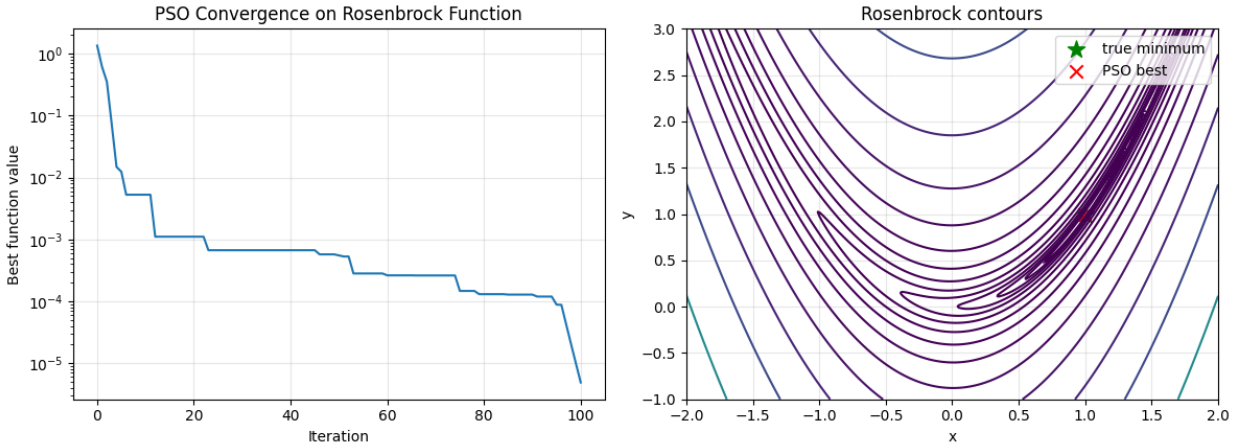
- Initial Solution: 30 particles
- Inertia Weight w = 0.70
- c1 = 1.50, c2 = 1.50
- Maximum Iterations: 100

Program Execution

```
Best Position   : [0.99778479 0.99557816]
Minimum Value   : 4.908494217485674e-06
Theoretical Min : [1, 1], f = 0
```

Execution Status: Successfully Executed

Convergence Graph



X-axis → Left: Iteration; Right: x

Y-axis → Left: Best function value (log scale); Right: y

Analysis

The best value falls by several orders of magnitude and the swarm lands close to the valley minimum at (1, 1). The Rosenbrock valley is easy to enter but slow to follow, which is why the curve flattens in steps.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Number of Particles	30	10	f = 4.9085e−06	f = 1.9917e−05	A smaller swarm covers less of the space and gives a larger final error.
2	Inertia Weight w	0.70	0.40	f = 4.9085e−06	f = 1.2869e−11	Lower inertia gave much stronger convergence to (1, 1) on this function.
3	Cognitive c1	1.50	2.50	f = 4.9085e−06	f = 2.3303e−04	A larger c1 pulls particles toward their own bests and slows swarm convergence.
4	Maximum Iterations	100	250	f = 4.9085e−06	f = 3.4593e−11	More iterations let the swarm refine the answer to about 1e−11.

EXPERIMENT NO. 09

L1, L2 and Elastic Net Regularization

1. Problem + Algorithm

Experiment No.: 11

Title: L1, L2 and Elastic Net Regularization

Aim: Demonstrate L1, L2 and Elastic Net regularization on a regression dataset.

Problem Statement: Fit linear regression with no penalty, an L2 (Ridge) penalty, an L1 (Lasso) penalty and an Elastic Net penalty on a 30-feature regression dataset and compare accuracy and coefficient sparsity.

Algorithm

- Generate the regression dataset and split into train and test sets.
- Standardize the features.
- Fit ordinary least squares as the baseline.
- Fit Ridge (L2), Lasso (L1) and Elastic Net models.
- Compute train R^2, test R^2 and test MSE.
- Count coefficients shrunk exactly to zero.
- Plot coefficients and sparsity for all models.
- Vary the penalty parameters and compare.

2. Program

Key parameter settings

```
X, y, true_coef = make_regression(n_samples=2000, n_features=30, n_informative=8,
                                  noise=25, coef=True, random_state=1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=1)

scaler = StandardScaler().fit(X_train)
X_train, X_test = scaler.transform(X_train), scaler.transform(X_test)

ridge_alpha = 10
lasso_alpha = 1
enet_alpha = 0.1
enet_l1_ratio = 0.5
```

Function definitions

```
def evaluate(model):
    model.fit(X_train, y_train)
    return dict(
        Train_R2=r2_score(y_train, model.predict(X_train)),
        Test_R2=r2_score(y_test, model.predict(X_test)),
        Test_MSE=mean_squared_error(y_test, model.predict(X_test)),
        Zero_Coefficients=int(np.sum(np.abs(model.coef_) < 1e-6)),
    )
```

Core algorithm

```
models = {
    "No Regularization": LinearRegression(),
    "L2 (Ridge)": Ridge(alpha=ridge_alpha),
    "L1 (Lasso)": Lasso(alpha=lasso_alpha),
    "Elastic Net": ElasticNet(alpha=enet_alpha, l1_ratio=enet_l1_ratio),
}
results = pd.DataFrame({name: evaluate(m) for name, m in models.items()}).T
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function: Minimize $\|y - Xw\|^2$ + penalty; L2 = $\alpha\|w\|_2^2$, L1 = $\alpha\|w\|_1$, Elastic Net = both

Search Interval: 2000 samples, 30 features (8 informative), noise = 25, 25% test split

Settings

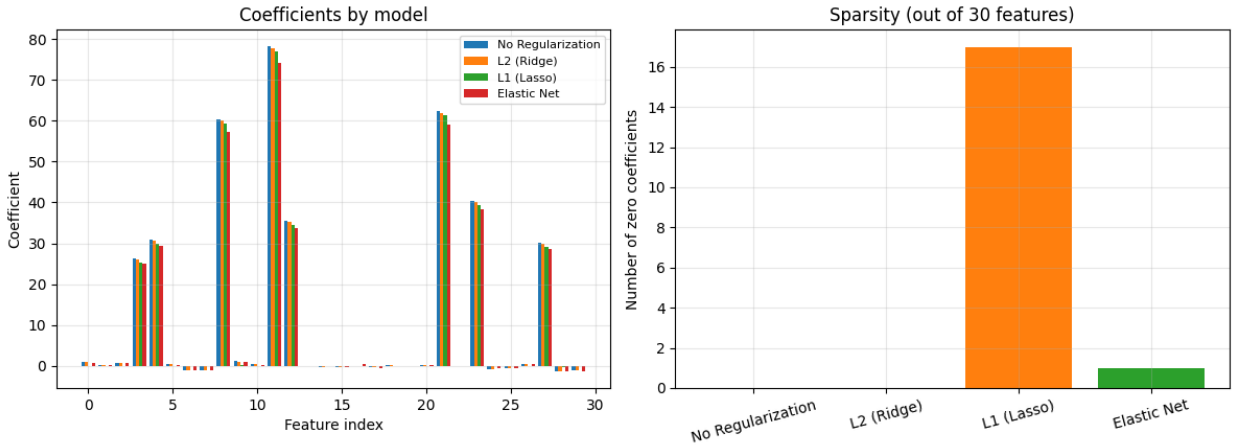
- Ridge α = 10
- Lasso α = 1
- Elastic Net α = 0.1, l1_ratio = 0.5

Program Execution

	Train_R2	Test_R2	Test_MSE	Zero_Coefficients
No Regularization	0.9654	0.9715	608.7544	0.0
L2 (Ridge)	0.9654	0.9713	612.5622	0.0
L1 (Lasso)	0.9645	0.9709	620.5647	17.0
Elastic Net	0.9628	0.9679	684.4403	1.0

Execution Status: Successfully Executed

Convergence Graph



X-axis → Left: Feature index; Right: Model

Y-axis → Left: Coefficient value; Right: Number of zero coefficients

Analysis

All models reach a test R^2 near 0.97. Ridge keeps every coefficient, only shrinking them, while Lasso removes 17 of the 22 uninformative features at almost no loss in accuracy. Elastic Net sits between the two.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Ridge alpha	10	1000	Test MSE = 612.56, zero = 0	Test MSE = 4368.35, zero = 0	Heavy L2 shrinkage underfits, and no coefficient becomes exactly zero.
2	Lasso alpha	1	20	Test MSE = 620.56, zero = 17	Test MSE = 4453.40, zero = 22	A stronger L1 penalty zeroes more features but starts removing informative ones, so error rises.
3	Elastic Net alpha	0.1	5	Test MSE = 684.44, zero = 1	Test MSE = 12104.79, zero = 18	Too strong a combined penalty underfits badly.
4	Elastic Net l1_ratio	0.5	0.9	Test MSE = 684.44, zero = 1	Test MSE = 615.32, zero = 2	A higher L1 share increased sparsity and lowered the error here.

EXPERIMENT NO. 10

Ant Colony Optimization

1. Problem + Algorithm

Experiment No.: 13

Title: Ant Colony Optimization

Aim: Implement Ant Colony Optimization (ACO) for finding a short tour through the given cities.

Problem Statement: Find a short closed tour through 20 cities A–T using pheromone trails and distance-based desirability.

Algorithm

- Initialize the distance matrix and pheromone trails.
- Calculate desirability ηij = 1 / dij.
- Calculate transition probabilities from pheromone and desirability.
- Construct tours for the ants.
- Calculate each tour length.
- Evaporate old pheromone.
- Calculate and add pheromone deposits.
- Repeat the process for further iterations.
- Return the best tour found.

2. Program

Key parameter settings

```
rng = np.random.default_rng(7)
n_cities = 20
city_names = list("ABCDEFGHIJKLMNOPQRST")
coords = rng.integers(0, 100, size=(n_cities, 2))
distance = np.linalg.norm(coords[:, None] - coords[None, :], axis=2)
eta = np.zeros_like(distance)
np.divide(1.0, distance, out=eta, where=distance > 0)

n_ants = 20
n_iterations = 100
alpha = 1
beta = 2
rho = 0.5
Q = 100
```

Function definitions

```
def tour_length(tour):
    return sum(distance[tour[i], tour[(i + 1) % n_cities]] for i in range(n_cities))


def build_tour(pheromone, alpha, beta, gen):
    start = gen.integers(n_cities)
    tour = [start]
    unvisited = [c for c in range(n_cities) if c != start]
    while unvisited:
        cur = tour[-1]
        weights = pheromone[cur, unvisited]**alpha * eta[cur, unvisited]**beta
        nxt = gen.choice(unvisited, p=weights / weights.sum())
        tour.append(nxt)
        unvisited.remove(nxt)
    return tour
```

Core algorithm

```
def aco(n_ants, n_iterations, alpha, beta, rho, Q, seed=42):
    gen = np.random.default_rng(seed)
    pheromone = np.ones((n_cities, n_cities))
    best_tour, best_len = None, np.inf
    history = []
    for _ in range(n_iterations):
        tours = [build_tour(pheromone, alpha, beta, gen) for _ in range(n_ants)]
        lengths = [tour_length(t) for t in tours]
        pheromone *= (1 - rho)
        for t, L in zip(tours, lengths):
            for i in range(n_cities):
                a, b = t[i], t[(i + 1) % n_cities]
                pheromone[a, b] += Q / L
                pheromone[b, a] += Q / L
        k = int(np.argmin(lengths))
        if lengths[k] < best_len:
            best_tour, best_len = tours[k], lengths[k]
        history.append(best_len)
    return best_tour, best_len, history
```

The complete executable program can be maintained digitally through Github.

3. Output + Analysis

Objective Function: Minimize total tour length Lk = Σ dij over the closed tour

Search Interval: 20 cities A–T with random coordinates in [0, 100] (seed 7)

Settings

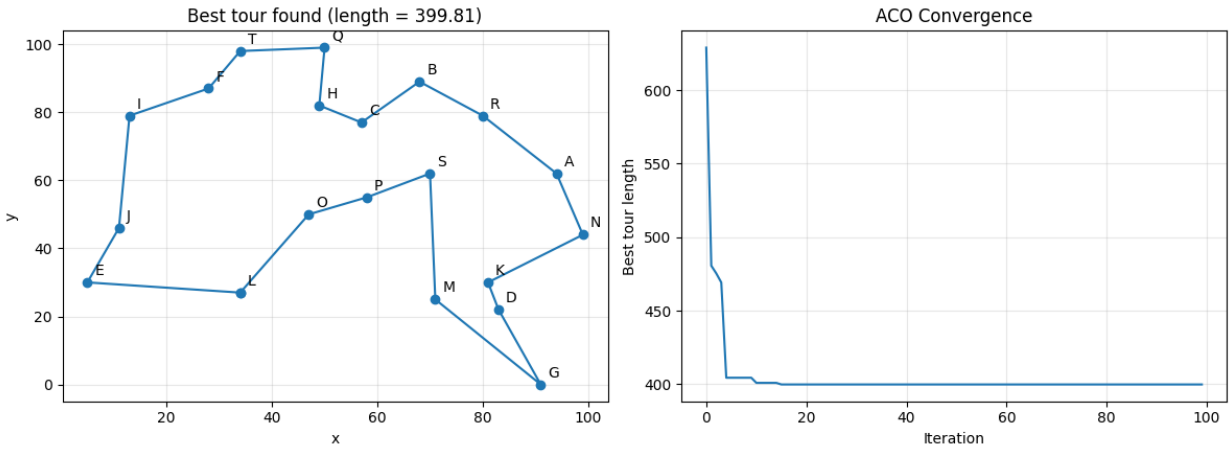
- Initial Solution: Pheromone τij = 1 on every edge
- α = 1, β = 2
- Evaporation Rate ρ = 0.50
- Q = 100, 20 ants, 100 iterations

Program Execution

```
Best Tour   : L -> E -> J -> I -> F -> T -> Q -> H -> C -> B -> R -> A -> N -> K -> D -> G -> M -> S -> P -> O
-> L
Best Length : 399.815
```

Execution Status: Successfully Executed

Convergence Graph



X-axis → Left: x coordinate; Right: Iteration

Y-axis → Left: y coordinate; Right: Best tour length

Analysis

Early iterations produce long tours because pheromone is uniform. As short edges collect more pheromone, ants choose them more often and the best tour length drops quickly, reaching 399.82. The same 20 cities give 419.75 with the GA in Experiment 07.

Parameter Modification

S. No.	Parameter	Original	Modified	Original Result	Modified Result	Observation
1	Pheromone Influence alpha	1	3	Length = 399.815	Length = 407.517	Over-weighting pheromone made ants follow early tours and ended in a longer tour.
2	Distance Influence beta	2	5	Length = 399.815	Length = 399.815	The same best tour was found; distance preference was already strong at β = 2.
3	Evaporation Rate rho	0.50	0.10	Length = 399.815	Length = 399.815	The same best tour was found with slower evaporation.
4	Deposit Constant Q	100	10	Length = 399.815	Length = 399.815	The same best tour was found; only the relative pheromone scale changed.